

Pràctica 7: Xarxa neuronal per a classificació de spam

Laura Rodríguez Cima – Víctor Navas Portella

Grau d'Estadística Aplicada

Curs 2025/2026

Objectiu

En aquesta pràctica construireu una xarxa neuronal per a un problema de **classificació de text**.

Treballarem amb un conjunt de dades de missatges SMS etiquetats com:

- **ham**: missatge normal
- **spam**: missatge brossa o publicitari

Objectiu de classificació

A partir del text d'un missatge, el model ha de predir si és **spam** o **no spam (ham)**.

Idea clau

A diferència de la pràctica anterior, aquí no partim d'imatges ni de variables numèriques directes: partim de **text**. Per poder entrenar una xarxa neuronal, primer cal transformar aquest text en vectors de nombres.

El dataset

Treballarem amb el fitxer `spam.csv`.

Cada fila del fitxer correspon a un missatge SMS i conté dues columnes principals:

- `v1`: etiqueta (ham o spam)
- `v2`: text cru del missatge

Exemple

- ham → "Ok lar... Joking wif u oni..."
- spam → "Free entry in 2 a wkly comp to win FA Cup..."

Important

Cada observació és un **missatge de text**, però una xarxa neuronal no pot treballar directament amb paraules. Necessitem convertir cada missatge en un vector numèric.

De text a variables numèriques

Per entrenar una xarxa neuronal, cada observació ha de ser un vector de nombres.
En aquesta pràctica utilitzarem una representació senzilla anomenada **Bag of Words**.

Idea general:

- construïm un vocabulari amb les paraules que apareixen als missatges
- cada paraula del vocabulari es converteix en una variable
- cada missatge es representa segons quines paraules hi apareixen

Exemple simple

Si el vocabulari fos:

[free, win, call, ok]

llavors el missatge:

'free call now'

es podria representar aproximadament com:

[1, 0, 1, 0]

Què fa la vectorització?

Quan vectoritzem el text, transformem una columna de missatges en una matriu numèrica.

Abans:

- fila 1 → I'm gonna be home soon
- fila 2 → Congratulations! You won a prize.

Després:

- fila 1 → vector de 0s i 1s
- fila 2 → un altre vector de 0s i 1s

Idea clau

La xarxa neuronal no veu frases, ni paraules, ni significats. Només veu vectors numèrics que representen el contingut dels missatges.

Què farem avui?

En aquesta pràctica seguirem els passos habituals en un problema de xarxes neuronals:

- carregar el conjunt de dades
- explorar alguns missatges
- transformar el text en variables numèriques
- preparar les dades per entrenar la xarxa
- construir el model
- entrenar-lo
- avaluar-ne el rendiment

Objectiu de la pràctica

Entendre com adaptar una xarxa neuronal a un problema de classificació de text.

Quina llibreria utilitzarem?

En aquesta pràctica treballarem amb:

- **pandas** per carregar i manipular les dades
- **scikit-learn** per vectoritzar el text
- **Keras / TensorFlow** per construir la xarxa neuronal

Per què és una bona opció aquí?

- Ens permet transformar text en variables numèriques de manera senzilla
- Podem reutilitzar gairebé el mateix esquema d'entrenament que a la pràctica anterior
- És una manera natural d'introduir xarxes neuronals en problemes de text

Imports necessaris

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

from sklearn.model_selection import train_test_split
from sklearn.feature_extraction.text import CountVectorizer
from sklearn.metrics import (
    accuracy_score,
    confusion_matrix,
    classification_report,
    ConfusionMatrixDisplay
)

from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Input, Dense
```


Pas 1: Carregar les dades

Comencem carregant el fitxer `spam.csv`.

```
data = pd.read_csv("spam.csv", encoding="latin-1")  
  
print(data.head())  
print(data.shape)  
print(data.columns)
```

Tasca

Observa:

- quantes files i columnes té el dataset
- quins noms de columna apareixen
- quines columnes semblen útils per al problema

Pas 2: Quedar-nos amb les columnes necessàries

El fitxer de Kaggle sovint inclou algunes columnes addicionals buides. Ens quedarem només amb les dues que necessitem.

```
data = data[["v1", "v2"]].copy()
data.columns = ["label", "text"]

print(data.head())
print(data["label"].value_counts())
```

Tasca

Comprova que:

- label conté les etiquetes
- text conté el missatge
- hi ha observacions de les dues classes

Pas 3: Explorar alguns missatges

Abans de transformar les dades, és útil mirar alguns exemples.

```
print("Exemples de missatges:")
print()

for i in range(5):
    print("Etiqueta:", data.loc[i, "label"])
    print("Text:", data.loc[i, "text"])
    print("-" * 50)
```

Tasca:

- Llegeix alguns missatges
- Intenta identificar quins trets podrien fer pensar que un missatge és spam

Pas 4: Convertir l'etiqueta a format numèric

La variable resposta és textual (ham / spam), però la xarxa necessita valors numèrics.

```
data["label_num"] = data["label"].map({  
    "ham": 0,  
    "spam": 1  
})  
  
print(data[["label", "label_num"]].head())
```

Interpretació

- 0: no spam
- 1: spam

Pas 5: Vectoritzar el text

Ara transformarem els missatges en una **matriu de variables numèriques**.

Farem servir `CountVectorizer()`, que:

- llegeix tots els missatges
- construeix un vocabulari
- crea una variable per a cada paraula del vocabulari
- representa cada missatge segons les paraules que hi apareixen

En aquesta pràctica utilitzem una representació molt senzilla:

- 1 si la paraula apareix al missatge, 0 si no apareix

Important

Aquesta és una manera **molt simple** de representar text. No tenim en compte:

- l'ordre de les paraules
- el significat
- la freqüència exacta
- moltes altres característiques que es podrien tenir en compte

Tot i ser simple, aquesta representació ja permet entrenar una xarxa neuronal.

Pas 5: Vectoritzar el text

Implementa el següent codi per construir la matriu d'entrada.

```
vectorizer = CountVectorizer(binary=True)

X = vectorizer.fit_transform(data["text"]).toarray()
y = data["label_num"].values

print("X shape:", X.shape)
print("y shape:", y.shape)
```

Tasca:

- Comprova quantes observacions tenim
- Comprova quantes variables s'han creat
- Pensa què vol dir que el nombre de variables sigui gran

Visualitzar com han quedat les dades

Podem inspeccionar algunes paraules del vocabulari i veure com queda la representació numèrica.

```
feature_names = vectorizer.get_feature_names_out()

print("Primeres 20 paraules del vocabulari:")
print(feature_names[:20])

print("\nPrimer vector d'entrada (primeres 30 components):")
print(X[0][:30])
```

Tasca:

- Observa algunes paraules del vocabulari
- Mira el vector d'una observació
- Comprova que els valors són 0 o 1

Interpretació de les dimensions

Si obteniu:

$$X \in \mathbb{R}^{5574 \times d}$$

això vol dir que tenim:

- 5574 missatges
- d variables numèriques, una per cada paraula del vocabulari seleccionat

I si:

$$y \in \mathbb{R}^{5574}$$

això vol dir que per a cada missatge tenim una etiqueta:

- 0: ham
- 1: spam

Important

A diferència de MNIST, aquí no tenim una estructura bidimensional com una imatge. Cada observació ja és directament un vector numèric, de manera que la xarxa el pot rebre directament com a entrada (no cal Flatten).

Pas 6: Separar dades d'entrenament i test

Ara separarem les dades en conjunt d'entrenament i conjunt de test.

```
# També guardem els textos originals per poder-los inspeccionar després
X_train, X_test, y_train, y_test, text_train, text_test = train_test_split(
    X,
    y,
    data["text"].values,
    test_size=0.2,
    random_state=123,
    stratify=y
)

print("X_train shape:", X_train.shape)
print("y_train shape:", y_train.shape)
print("X_test shape:", X_test.shape)
print("y_test shape:", y_test.shape)
```

Per què usem stratify=y?

Per mantenir aproximadament la mateixa proporció de missatges ham i spam en train i test.

Pas 7: Definir l'arquitectura i construir el model

Ara heu de definir la xarxa neuronal per a la classificació de missatges.

Idea general:

$$d \rightarrow \text{capes ocultes} \rightarrow 1$$

Interpretació:

- Entrada: vector de longitud d
- Una o més capes ocultes amb activació ReLU
- Capa de sortida: 1 neurona amb activació _

Important

Aquest és un problema de classificació binària, per tant la sortida no ha de tenir moltes neurones sinó una única neurona que retorni una probabilitat.

Pas 7: Definir l'arquitectura i construir el model

Tasca: Construeix una xarxa neuronal per a aquest problema.

Pots utilitzar pràctiques anteriors com a exemple, però adapta l'arquitectura a aquest cas.

```
model = Sequential([  
    .....  
    .....  
    .....  
])
```

Tasca: Utilitza `model.summary()` per verificar que l'arquitectura és correcta.

Pas 8: Configurar l'entrenament

Un cop definida la xarxa, cal indicar **com s'entrenarà**.

Elements principals que heu de definir:

- **Optimitzador:** com s'actualitzen els pesos
- **Funció de pèrdua:** què volem minimitzar
- **Mètrica:** com mesurem el rendiment

Tasca

Configura l'entrenament del model. En aquest cas, pensa quina funció de pèrdua és adequada per a classificació binària.

Pas 8: Configurar l'entrenament

Tasca: Completa el codi.

```
model.compile(  
    .....  
    .....  
    .....  
)
```

Pas 9: Entrenar el model

Un cop definit i configurat el model, cal entrenar-lo amb les dades del conjunt de `train`.

Durant l'entrenament, la xarxa:

- calcula les prediccions
- mesura l'error
- ajusta els pesos

Paràmetres principals:

- `epochs`: nombre de vegades que recorrem les dades d'entrenament
- `batch_size`: mida dels blocs de dades
- `validation_split`: percentatge reservat per validació
- `verbose`: informació mostrada per pantalla

Pas 9: Entrenar el model

Tasca: Entrena la xarxa.

```
entrenament = model.fit(  
    .....  
    .....  
    .....  
    .....  
    .....  
    .....  
    .....  
)
```

Tasca: Observeu durant l'entrenament:

- com evoluciona la pèrdua (loss)
- com evoluciona l'accuracy

Visualització de l'entrenament

Podem representar l'evolució de la pèrdua i de l'accuracy.

```
def plot_entrenament(entrenament):  
    plt.figure(figsize=(12,4))  
  
    plt.subplot(1,2,1)  
    plt.plot(entrenament.history['loss'], label='train')  
    plt.plot(entrenament.history['val_loss'], label='validation')  
    plt.title('Loss')  
    plt.legend()  
  
    plt.subplot(1,2,2)  
    plt.plot(entrenament.history['accuracy'], label='train')  
    plt.plot(entrenament.history['val_accuracy'], label='validation')  
    plt.title('Accuracy')  
    plt.legend()  
  
    plt.tight_layout()  
    plt.show()  
  
plot_entrenament(entrenament)
```


Pas 10: Avaluar el model

Un cop entrenat el model, cal avaluar-lo sobre dades noves.

En aquest cas, utilitzem el conjunt de test:

- Són dades que el model no ha vist durant l'entrenament
- Serveixen per mesurar la capacitat de generalització

Què fa `model.evaluate(...)`?

- Fa prediccions sobre X_{test}
- Compara aquestes prediccions amb les etiquetes reals
- Calcula la funció de pèrdua i les mètriques

Tasca: Avalua el model.

```
loss_test, acc_test = .....  
print("Test loss:", loss_test)  
print("Test accuracy:", acc_test)
```

Pas 11: Obtenir prediccions

Un cop entrenada la xarxa, podem obtenir la probabilitat que un missatge sigui spam.

- `model.predict(X_test)` retorna, per a cada missatge, una probabilitat entre 0 i 1
- Aquesta probabilitat indica com de probable és que el missatge sigui spam

Tasca: Obteniu les probabilitats i la classe predita per al conjunt de test.

```
# Obtenir probabilitats
y_prob = .....

# Convertir probabilitats en classes
y_pred = (y_prob >= 0.5).astype(int).ravel()

print("Primeres 10 probabilitats:")
print(y_prob[:10].ravel())

print("\nPrimeres 10 prediccions:")
print(y_pred[:10])
```

Matriu de confusió

Un cop tenim les prediccions del model, podem resumir els resultats amb una **matriu de confusió**, que és una taula que compara les etiquetes reals i les etiquetes predites pel model.

```
cm = confusion_matrix(y_test, y_pred)

disp = ConfusionMatrixDisplay(confusion_matrix=cm, display_labels=["ham", "spam"])
disp.plot(cmap="Blues")
plt.title("Matriu de confusi ")
plt.show()
```

Com s'interpreta?

- la diagonal principal conté les classificacions correctes
- els valors fora de la diagonal corresponen a errors

Tasca: Observa si el model comet més errors classificant spam com ham o a l'inrevés.

Visualitzar prediccions del model

Tasca: Utilitza la funció següent per comparar etiqueta real i predicció en alguns missatges del test.

```
def mostrar_prediccions(texts, y_true, y_pred, y_prob, n=10):  
    for i in range(n):  
        etiqueta_real = "spam" if y_true[i] == 1 else "ham"  
        etiqueta_pred = "spam" if y_pred[i] == 1 else "ham"  
  
        print(f"Text: {texts[i]}")  
        print(f"Real: {etiqueta_real}")  
        print(f"Predicci : {etiqueta_pred}")  
        print(f"Prob(spam): {float(y_prob[i]):.4f}")  
        print("-" * 100)  
  
mostrar_prediccions(text_test, y_test, y_pred, y_prob, n=10)
```

Visualitzar errors de classificació

Tasca: Inspecciona quins casos s'han classificat malament.

```
def mostrar_errors(texts, y_true, y_pred, y_prob, n=10):
    errors = np.where(y_true != y_pred)[0]

    print("Nombre total d'errors:", len(errors))
    print()

    for idx in errors[:n]:
        etiqueta_real = "spam" if y_true[idx] == 1 else "ham"
        etiqueta_pred = "spam" if y_pred[idx] == 1 else "ham"

        print(f"Text: {texts[idx]}")
        print(f"Real: {etiqueta_real}")
        print(f"Predicci : {etiqueta_pred}")
        print(f"Prob(spam): {float(y_prob[idx]):.4f}")
        print("-" * 100)

mostrar_errors(text_test, y_test, y_pred, y_prob, n=10)
```

Provar la xarxa amb missatges nous

Tasca: Ara podeu provar el model amb missatges escrits per vosaltres mateixos.

L'objectiu és veure com la xarxa assigna una probabilitat de ser spam a missatges nous que no formen part del conjunt de test.

```
# Prova amb un missatge nou

nous_missatges = [
    "Congratulations! You have won a free ticket. Call now!",
    "Are we still meeting at 7 tonight?",
    "URGENT! claim your prize now",
    "Ok see you later"
]

X_nou = vectorizer.transform(nous_missatges).toarray()
prob_nou = model.predict(X_nou, verbose=0)
pred_nou = (prob_nou >= 0.5).astype(int).ravel()

for i, msg in enumerate(nous_missatges):
    etiqueta = "spam" if pred_nou[i] == 1 else "ham"
    print(f"Missatge: {msg}")
    print(f"Prob(spam): {float(prob_nou[i]):.4f}")
    print(f"Predicci : {etiqueta}")
    print("-" * 100)
```

Tasca: Experimenta amb diferents opcions igual que vas fer en la pràctica anterior. Pots recuperar la guia de l'experimentació de la pràctica 6.

Reflexions finals

- Per què cal transformar el text en variables numèriques?
- Quina diferència hi ha entre `evaluate` i `predict`?
- Per què la xarxa retorna una probabilitat i no directament una classe?
- Quins errors comet més sovint el model?
- Quines limitacions té representar el text només amb Bag of Words?